

教學實作：擴充 SG90 伺服馬達功能

本章節實作一個「具備物理空間感」的伺服馬達控制功能。

步驟 1：修改硬體定義提示詞 (Devices Definition)

讓 AI 認識伺服馬達及其物理規則。

- SG90 Servo Motor: Pin 2 (Control range: 0 – 180 degrees).
- Physical Rules: Increasing values correspond to "clockwise" rotation, while decreasing values correspond to "counter-clockwise" rotation.

步驟 2：定義工具 JSON 規範 (Tools Definition)

在 `tools_definition` 中加入 `/servo` 指令的格式規範。

// If the system determines that the user expects assistance in controlling a servo motor, it must return only this valid JSON object:

```
{
  "type": "tool_call",
  "method": "/servo",
  "params": {
    "pin": "<Device pin number. If the user explicitly specifies a pin number, use that number. If the user does not specify which device pin to control, first ask the user in normal conversational reply and wait for the user's answer before filling this value. If multiple pins are mentioned, choose only the first one in order of appearance.>",
    "angle": "<Desired absolute angle from 0 to 180>"
  }
}
```

步驟 3：底層 C++ 程式碼實作

在專案中新增 tool_servo 驅動函數。

```
#include <AmebaServo.h>
AmebaServo servo2;

// Control a servo motor's position by specifying a target angle.
// This function supports precise physical movement for actuators like the SG90 servo connected to GPIO pins.
String tool_servo(AmebaServo &servo, int pin, int angle) {
    if (!servo.attached())
        servo.attach(pin);
    angle = constrain(angle, 0, 180);
    servo.write(angle);

    return "Servo on pin " + String(pin) + " moved to " + String(angle) + " degrees.";
}

void setup() {
    servo2.attach(2);
}
```

步驟 4：更新工具分發器邏輯 (useTools)

將 /servo 方法註冊進系統的分發流程。

```
void useTools(String command, JsonObject params) {
    if (command == " ... ") {
        // ...
    }
    else if (command == "/servo") {
        int pin = params["pin"].as<int>();
        int angle = params["angle"].as<int>();

        // Execute hardware action
        String response = "";
        if (pin==2)
            response = tool_servo(servo2, pin, angle);
        else
            response = "The servo pin " + String(pin) + " is not defined";

        // Update conversation history (Memory Persistence)
        historical_messages += buildHistoricalData("user", command);
        historical_messages += buildHistoricalData("model", response);

        // Save to SD card (Persistence)
        storeHistoricalMessagesToFile();

        // Send a follow-up request to Gemini after completing a tool execution step.
        // The purpose is to let the model evaluate the current system state,
        // decide whether the task is finished, or whether another tool action is still required.
```

```

//
// This acts as a "decision checkpoint" in the tool-calling pipeline:
// - If workflow is incomplete → Gemini should return a tool_call JSON for next action
// - If workflow is complete → Gemini should respond in natural language
response = Gemini_chat_request(
    "Analyze the execution result and determine whether the workflow is complete.\n\n"
    "If additional hardware actions are strictly required to complete the request, "
    "return ONLY a valid tool_call JSON.\n"
    "Otherwise, respond naturally in the user's language.",
    1
);

// Parse and route Gemini response to the tool execution layer.
// This function is responsible for:
// 1. Detecting whether the response contains a valid tool_call JSON
// 2. Extracting method and parameters if tool execution is required
// 3. Dispatching the command to the corresponding hardware / system handler (useTools)
// 4. Otherwise, treating it as a normal chat message and sending it to Telegram or user output
gemini_router(response);
}

```